BillSplit — Project Documentation

Full End-to-End System Architecture, High-Level Design (HLD), Data Flow Diagrams (DFD), Database Schema, Technology Stack Justification & Viva Voce Preparation Guide

Project Name: BillSplit	**Target Surface:** Android (Java & XML)
Backend: Node.js, Express.js, MongoDB	**Networking:** Retrofit 2 & Gson
Architecture: 3-Tier MVC / RESTful API	**Financial Precision:** Integer Paise Engine

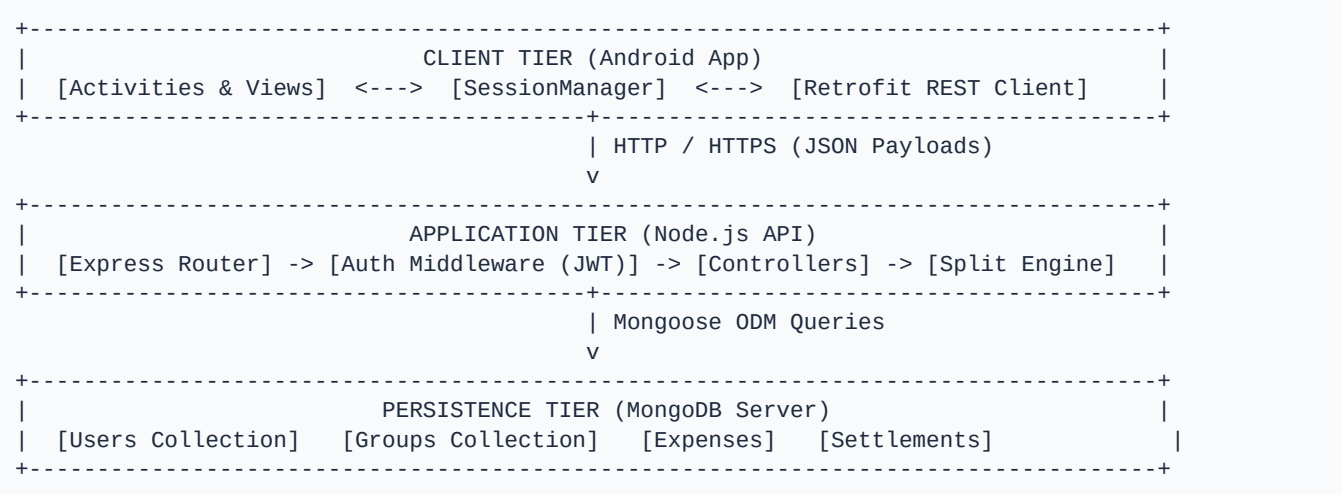
1. Executive Summary & Problem Statement

Managing shared expenses among roommates, flats, or group trips is a common problem. Traditional manual expense tracking using paper or spreadsheets often leads to mathematical errors, forgotten debts, payment disputes, and difficulty calculating exact multi-person splits.

BillSplit is an end-to-end multi-user expense splitter application. It enables users to record shared household costs, compute exact equal and custom percentage/amount splits without rounding errors, track net individual balances, simplify debt transactions, record payment settlements, and view detailed monthly spending analytics.

2. High-Level System Design (HLD)

The system follows a classic 3-Tier Distributed Client-Server Architecture:



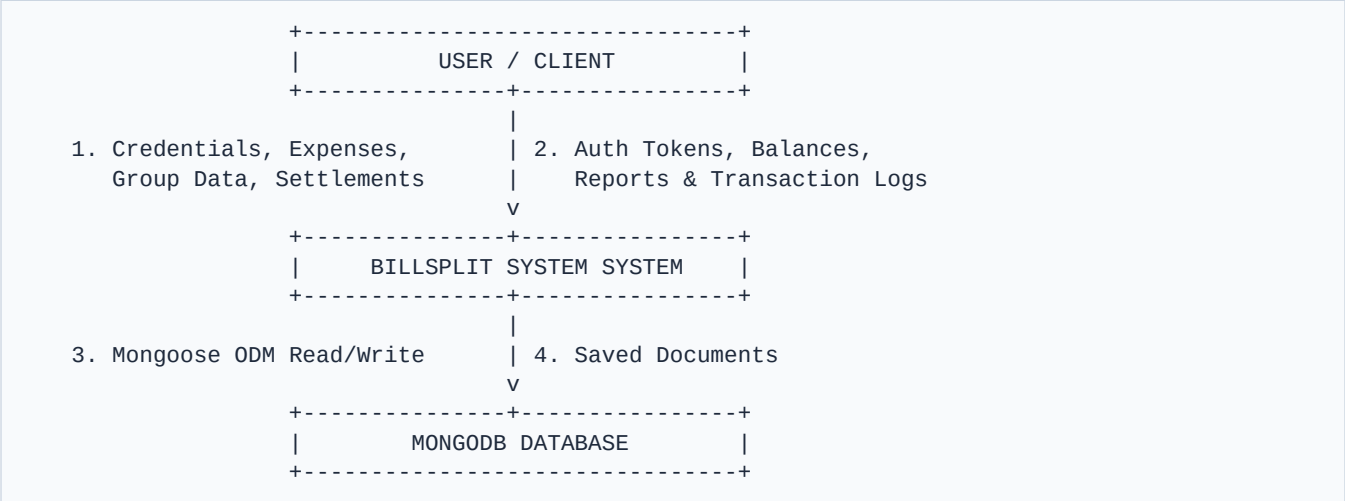
Component Description:

- **Presentation Layer (Android App)**: Built with Java and XML. Includes activities for Login, Register, Dashboard, Add/Edit Expense, Group Details, Settlement, Reports, and Profile.

- **Networking Layer**: Retrofit 2 handles async HTTP network calls with Gson converter for automatic JSON serialization/deserialization.
- **Session Layer**: SessionManager persists JWT authentication tokens and user profile state securely in Android SharedPreferences.
- **Application Layer (Node.js & Express)**: Receives API requests, enforces authentication middleware, validates request bodies, and executes business logic.
- **Database Layer (MongoDB)**: Document-oriented MongoDB database accessed via Mongoose ODM models.

3. Data Flow Diagrams (DFD)

DFD Level 0 — Context Diagram:



DFD Level 1 — Macro Process Diagram:

- **Process 1.0 (User Auth)**: User enters name, email, password -> AuthController validates -> BCrypt hashes password -> JWT token generated -> Token returned to Client.
- **Process 2.0 (Group Management)**: User creates group -> GroupController validates members -> Saves Group Document -> Updates Active Group state.
- **Process 3.0 (Expense & Split Calculation)**: User inputs expense details and split method -> SplitCalculator computes paise shares -> Saves Expense Document -> Recalculates Net Member Balances.
- **Process 4.0 (Settlement Processing)**: Payer selects receiver and amount -> Settlement recorded -> Deducts outstanding debt -> Updates Dashboard balances instantly.
- **Process 5.0 (Reports Generation)**: Fetches group expenses -> Category totals & member spending aggregated -> Returns analytics summary.

4. Technology Stack Justification

Technology	Role / Purpose	Why It Was Chosen
Android Java & XML	Mobile Frontend UI	Robust, industry-standard, backward compatible across Android API 24-36 without Kotlin runtime overhead.
Retrofit 2 & Gson	HTTP Network Client	Type-safe HTTP requests, automatic JSON-to-Java object conversion, asynchronous background execution.
SharedPreferences	Local Session Storage	Lightweight, encrypted, key-value storage ideal for persisting JWT auth tokens and active group settings.
Node.js & Express	REST API Backend	Non-blocking asynchronous I/O, rapid REST API development, native JSON handling.
MongoDB & Mongoose	NoSQL Database & ODM	Flexible document schema ideal for storing dynamic arrays of group members and expense split objects.
JWT (JSON Web Token)	Authentication Token	Stateless, secure, scalable session token passed in HTTP Authorization headers.
BCrypt.js	Password Hashing	Industry-standard salted password hashing algorithm preventing plain-text credential leaks.

5. Financial Engine & Algorithmic Design

1. Integer Paise Representation:
Floating-point representation (e.g., float / double) suffers from binary representation precision loss (e.g., 0.1 + 0.2 = 0.30000000000000004). To eliminate rounding errors, all monetary amounts are stored as integer **paise** (■1.00 = 100 paise). Formatting to Rupees occurs solely in the UI display layer.

2. Equal Split Remainder Distribution:
When total paise cannot be divided evenly (e.g. ■100.00 / 3 members = 10000 paise / 3 = 3333 paise with 1 paise remainder), the base share is 3333 paise, and the 1 paise remainder is distributed to the first member. Result: Member 1 = ■33.34, Member 2 = ■33.33, Member 3 = ■33.33. Total sum = ■100.00 exactly.

3. Simplified Debts Algorithm (Greedy Settlement Minimization):
Given net balances for group members, the system separates debtors (net balance < 0) and creditors (net balance > 0). It greedily pairs the largest debtor with the largest creditor, creating a minimal settlement transaction until all balances reach zero. This reduces unnecessary group payments.

6. Viva Voce Examination — Questions & Model Answers

Master these questions and responses for your viva voce exam:

Q1: What is the main objective of the BillSplit project?

BillSplit is designed to streamline shared household and trip expense tracking among roommates and groups. It eliminates manual calculation errors, provides paise-exact expense splitting, calculates net balances, minimizes debt transactions, and records payment settlements in real time.

Q2: Why did you choose Java and XML for Android instead of Kotlin or Jetpack Compose?

Java and XML offer rock-solid stability, standard Android framework compatibility, explicit memory lifecycle management, and lightweight compilation across older and newer Android devices without additional Kotlin runtime dependencies.

Q3: Why did you store monetary values in integer paise instead of floating-point numbers?

Floating-point data types (float/double) suffer from binary IEEE 754 precision loss during addition/subtraction. Storing currency in integer paise guarantees exact arithmetic, ensuring that the sum of all individual split shares equals the total expense amount down to the single paise.

Q4: How does your Equal Split algorithm handle remainder paise?

When total paise is divided by the member count, integer division yields a base share and a remainder (remainder = totalPaise % memberCount). The algorithm adds 1 paise to the first 'remainder' members, ensuring sum(shares) == totalAmountPaise exactly.

Q5: Explain your Simplified Debts algorithm.

The algorithm computes net balance for every member (total paid minus total share plus settlements). Members are separated into Debtors (net balance < 0) and Creditors (net balance > 0). It iteratively matches the largest debtor with the largest creditor, creating a single direct debt transaction for min(debt, credit), reducing total group transactions.

Q6: Why did you choose Node.js, Express, and MongoDB for the backend?

Node.js with Express provides non-blocking asynchronous event-driven I/O ideal for REST APIs. MongoDB with Mongoose ODM offers a flexible document schema perfectly suited for dynamic JSON arrays of group members and individual split records.

Q7: How is authentication implemented in BillSplit?

Authentication uses JWT (JSON Web Tokens) and BCrypt.js. When a user registers or logs in, BCrypt hashes/verifies the password. Upon successful validation, the server returns a signed JWT token, which the Android client stores in SessionManager (SharedPreferences) and attaches to subsequent HTTP Authorization headers.

Q8: How does Retrofit handle networking in the Android app?

Retrofit 2 is a type-safe HTTP client. It uses Gson Converter to automatically serialize Java objects to JSON request bodies and deserialize backend JSON responses into Java model classes asynchronously on background threads.

Q9: What is the difference between an 'Expense' and a 'Settlement'?

An Expense represents a shared cost (e.g. Rent, Groceries, Hotel) which increases group spending and splits costs among members. A Settlement represents a payment transaction where a debtor transfers money directly to a creditor to clear outstanding balance.

Q10: How does the Android app connect to the local backend server during development?

On Android Emulators, '10.0.2.2' bridges to host localhost. On physical mobile devices, ADB Reverse Port Forwarding ('adb reverse tcp:5000 tcp:5000') or the computer's local Wi-Fi IP address ('http://10.194.36.146:5000/') routes HTTP requests directly over USB or Wi-Fi.

Q11: How do you handle email case-sensitivity during login/registration?

Both the Android client and the backend authController normalize email strings by stripping whitespace (.trim()) and converting text to lowercase (.toLowerCase()) prior to MongoDB queries and BCrypt checks.

Q12: How do you prevent keyboard focus loss when typing custom split amounts?

In AddExpenseActivity, setupCustomInputs() checks a mode key (currentCustomMode) so it only inflates member EditText fields when member count or split mode changes. This prevents re-inflating views on every keystroke, maintaining focus and smooth typing.